

Módulo 12 – Capítulo 10 – Sección 01

Rúbrica del proyecto final: criterios técnicos de evaluación por componente

El Capítulo 1 de este módulo presentó los criterios de éxito del sistema integrador como los destinos que guiarían cada decisión técnica: RAGAS faithfulness ≥ 0.85 en producción, task completion rate $\geq 75\%$, latencia P95 $< 3s$ bajo carga, y tasa de bypass de controles de seguridad $< 5\%$. A lo largo de los nueve capítulos siguientes, esos criterios funcionaron como la brújula que orientó las decisiones de diseño — por qué se eligió 512 tokens como tamaño de chunk, por qué el reranker Cohere permanece en el pipeline a pesar de sus 280ms de latencia adicional, por qué el intent classifier se configura para fallar de forma segura ante la incertidumbre. El Capítulo 10 cierra el ciclo: la rúbrica del proyecto final formaliza esos criterios como los instrumentos de medición con los que el sistema construido se evalúa objetivamente.

La rúbrica no es una lista de características a verificar — es un instrumento de medición con criterios cuantitativos que pueden verificarse con scripts automatizados, no con juicio subjetivo. La diferencia es importante: "la seguridad está implementada" es una afirmación que no puede verificarse de forma consistente; "la tasa de bypass en el red teaming de 50 ataques es $< 5\%$ " sí puede. Esta cuantificación tiene una consecuencia directa en la evaluación: dos sistemas con el mismo umbral de faithfulness son comparables; dos sistemas donde uno "tiene seguridad" y el otro "tiene una postura de seguridad razonablemente madura" no lo son. La rúbrica cuantitativa es la que produce evaluaciones reproducibles.

Las seis dimensiones de evaluación cubren el sistema completo. La dimensión de **pipeline RAG** evalúa la calidad técnica del retrieval: RAGAS faithfulness ≥ 0.82 sobre el golden dataset completo (el umbral de CI, no el de producción — el evaluador del proyecto usa el golden dataset que el equipo construyó, no el tráfico de producción), búsqueda híbrida implementada con RRF (verificable en el código del retrieval pipeline), reranking activo con umbral configurable, y pipeline de ingesta con deduplicación verificada con el test de ingesta de documentos duplicados. Un faithfulness de 0.81 pasa el umbral de producción pero no el

de CI — en la rúbrica del proyecto, se usa el umbral de CI porque es el que el golden dataset del evaluador puede verificar de forma reproducible.

La dimensión **agéntica** evalúa la implementación del agente ReAct con LangGraph: estado persistente con `MemorySaver` o `SqliteSaver`, mínimo tres herramientas con schemas Pydantic tipados, `max_iterations` configurado como guardia de bucle infinito, y task completion rate $\geq 75\%$ medido sobre el golden dataset de escenarios agénticos (los 20 escenarios de prueba incluyendo los de fallo: retrieval vacío, timeout, ambigüedad de consulta).

La dimensión de **seguridad** evalúa la implementación de ADR-004: threat model STRIDE documentado con las ocho amenazas del sistema, mínimo tres controles implementados con referencia al código (el intent classifier de prompt injection, el RBAC a nivel de Qdrant y la validación de outputs con filtro de PII son los tres mínimos), y red teaming documentado con mínimo 20 ataques de al menos tres categorías diferentes, con resultados: número de ataques intentados, número mitigados, tasa de bypass final.

Criterios de evaluación por dimensión

- **Pipeline RAG:** RAGAS faithfulness ≥ 0.82 en golden dataset, búsqueda híbrida BM25+densas con RRF, reranking Cohere activo, ingesta con deduplicación verificada; verificable con script de evaluación autónomo.
- **Sistema agéntico:** ReAct con LangGraph, estado persistente implementado, ≥ 3 herramientas con schemas Pydantic, `max_iterations` configurado, task completion rate $\geq 75\%$ en golden dataset de 20 escenarios incluyendo casos de fallo.
- **Seguridad:** ADR-004 con STRIDE documentado, ≥ 3 controles implementados con referencia al código, red teaming ≥ 20 ataques en ≥ 3 categorías con tasa de bypass $< 5\%$; bypass $> 5\%$ es criterio de no aprobación de la dimensión.
- **MLOps:** Dockerfile multi-stage (usuario no-root, health check, hash de dependencias), pipeline CI/CD con ≥ 4 etapas incluyendo gate de evaluación RAGAS, despliegue reproducible con configuración de producción, secrets no presentes en el código.
- **Observabilidad:** OpenTelemetry instrumentado en ≥ 5 etapas del pipeline, dashboard con ≥ 5 paneles incluyendo métricas RAGAS y costo, ≥ 3 alertas configuradas con runbook_url referenciado, drift monitoring implementado.
- **Documentación:** README con setup funcional verificado por un revisor externo al equipo, API documentada con OpenAPI incluyendo ejemplos de borde, runbook con ≥ 3 incidentes, guía de contribución con ≥ 1 extensión documentada.

Nota del Arquitecto: La rúbrica tiene deliberadamente un criterio de no aprobación automática en la dimensión de seguridad: tasa de bypass > 5% en el red teaming. Las otras dimensiones permiten evaluación Parcial con deficiencias documentadas — un sistema con faithfulness 0.79 que documenta el gap y su causa puede obtener evaluación Parcial. Pero un sistema con tasa de bypass del 10% en los controles de seguridad no puede considerarse "parcialmente seguro" de la misma manera: si el 10% de los ataques de prompt injection pasan, el sistema no puede desplegarse en un entorno donde esos ataques son reales. Este criterio absoluto en seguridad refleja la diferencia entre una métrica de calidad (donde el degraded gracefully tiene valor) y una condición de correctitud (donde el sistema falla en una dimensión no negociable).

La rúbrica del proyecto final no es el criterio que el equipo espera que el evaluador aplique — es el criterio que el equipo debe aplicar sobre su propio sistema antes de presentarlo. Un equipo que llega a la evaluación sin haber ejecutado el script de evaluación RAGAS, sin haber completado el red teaming de 20 ataques, o sin haber verificado que el README funciona en una máquina limpia, no está enviando un sistema para evaluación — está enviando un borrador. La rúbrica como práctica de auto-evaluación es parte de la competencia que el módulo desarrolla.

Para recordar: Una rúbrica con criterios cuantitativos verificables convierte la evaluación del sistema en un proceso reproducible, no en una opinión. Los mismos criterios que el evaluador aplica son los que el equipo debe aplicar en el checklist de producción del Capítulo 10-02.

Módulo 12 – Capítulo 10 – Sección 02

Checklist de producción: verificaciones obligatorias antes de considerar el sistema listo

El primer despliegue de un sistema de IA a producción es el momento en que las suposiciones de diseño se confrontan con la realidad: los usuarios hacen preguntas que el golden dataset no cubría, los patrones de carga son diferentes de los que el benchmark de desarrollo modeló, y los proveedores de dependencias tienen comportamientos en producción que no se observaron en staging. El checklist de producción no elimina esas sorpresas — las sorpresas son inevitables en cualquier sistema nuevo. Lo que el checklist garantiza es que el equipo llega al primer despliegue sin las categorías de problemas que son completamente prevenibles: un secreto en el código, una imagen Docker con CVEs críticos, o un runbook que nadie revisó antes de que el primer incidente lo necesitara.

Un checklist de producción tiene valor solo cuando se trata como un contrato, no como una formalidad. La diferencia se observa en la actitud del equipo ante los ítems que no pueden completar antes del despliegue: si el checklist es un contrato, los ítems incompletos son bloqueadores del despliegue y requieren una decisión explícita de aceptar el riesgo documentada en el ADR o en el post-mortem preventivo; si el checklist es una formalidad, los ítems incompletos se marcan como "en progreso" y se planean para la siguiente semana sin bloquear el despliegue. La segunda actitud produce exactamente el tipo de incidentes post-despliegue que el checklist estaba diseñado para prevenir.

Las 30 verificaciones del checklist se agrupan en seis categorías que cubren el sistema de forma exhaustiva. La categoría de **seguridad** es la primera porque sus ítems son los más costosos de remediar post-despliegue: detectar que hay un secreto en el historial de Git después de desplegar requiere revocar el secreto, regenerarlo, rotar todas las dependencias que lo usan, y realizar un audit para determinar si el secreto fue expuesto durante el tiempo que estuvo en el repositorio. `detect-secrets scan` como parte del pipeline CI es la forma correcta de prevenir esto — el checklist verifica que el scanner está activo, no que el equipo recuerda ejecutarlo manualmente.

La categoría de **calidad** verifica que el sistema cumple los umbrales de la rúbrica antes del despliegue: RAGAS faithfulness ≥ 0.82 en el golden dataset completo, task completion rate $\geq 75\%$ en los 20 escenarios agénticos, y pipeline CI/CD en verde con el gate de evaluación activo. El ítem más importante de esta categoría no es el faithfulness — es la verificación de que el golden dataset está actualizado y que el evaluador RAGAS del CI está usando la versión correcta del dataset. Un faithfulness de 0.87 calculado con un golden dataset de 50 pares obsoletos que no cubren las categorías de queries de producción actuales tiene mucho menos valor que un faithfulness de 0.83 calculado con un golden dataset de 200 pares recientemente anotados.

La categoría de **operabilidad** verifica las propiedades que el sistema necesita para ser operado de forma segura: `kubectl exec -it <pod> -- curl -s http://localhost:8000/health` responde en menos de 200ms (si el health check tarda más, el liveness probe de Kubernetes puede reiniciar el pod durante operación normal), graceful shutdown verificado enviando `SIGTERM` al proceso y confirmando que las requests en vuelo se completan antes del shutdown (sin graceful shutdown, el despliegue de una nueva versión puede producir errores 502 mientras el pod antiguo se detiene), y restart automático verificado con `kubectl kill pod <pod-name>` para confirmar que Kubernetes levanta un nuevo pod dentro del tiempo configurado.

Checklist de producción por categoría

- **Seguridad (8 ítems):** `detect-secrets scan` sin resultados, Trivy o CVEs críticos en imagen Docker, JWT validación verificada con token expirado y token con firma inválida, rate limiting activo verificado con script de carga, red teaming completado con bypass $< 5\%$, secrets en AWS Secrets Manager (no en ConfigMaps ni en código), `.env` excluido del repositorio con `.gitignore`, RBAC verificado con usuario sin permisos intentando acceder a documentos restringidos.
- **Calidad (6 ítems):** RAGAS faithfulness ≥ 0.82 en golden dataset actualizado (fecha de última anotación < 30 días), task completion rate $\geq 75\%$ en 20 escenarios agénticos, pipeline CI/CD verde con gate de evaluación activo, test de regresión pasando en staging, golden dataset versionado en repositorio con fecha y versión del modelo evaluador.
- **Rendimiento (5 ítems):** latencia P95 < 3000 ms bajo carga de 50 usuarios simultáneos (verificado con Locust), throughput ≥ 20 req/s sin degradación de calidad, costo por request documentado y dentro del presupuesto mensual proyectado, memoria del pod estable sin memory leak bajo carga sostenida de 30 minutos.

- **Operabilidad (6 ítems):** health check < 200ms, graceful shutdown verificado manualmente, restart automático verificado, recursos del pod (CPU/memoria request y limit) calibrados contra el perfil de carga real, HPA configurado si el sistema requiere escalado horizontal.
- **Observabilidad (3 ítems):** trazas visibles en Grafana Tempo para al menos 3 tipos de request, alertas CRITICAL configuradas con runbook_url apuntando a runbook existente, dashboard operativo accesible con datos del día actual.
- **Documentación (2 ítems):** README verificado en máquina limpia por un ingeniero que no escribió el sistema (setup en < 30 min), API documentada con `curl` que devuelve la respuesta de ejemplo documentada.

Nota del Arquitecto: Los dos ítems de documentación en la categoría de Documentación son deceptivamente simples — parecen triviales comparados con los 8 ítems de Seguridad. En la práctica, son frecuentemente los que más sorpresas producen. El README que el ingeniero autor verifica en su propia máquina (donde ya tiene todas las dependencias instaladas, las credenciales configuradas y el contexto mental del sistema) raramente reproduce las condiciones de un nuevo ingeniero en una máquina limpia. He visto READMEs que afirman que el setup toma 15 minutos y en una máquina limpia toman 90 minutos porque hay un paso que asume que Docker está instalado sin mencionarlo, y una dependencia del sistema (libssl-dev en Ubuntu) que no aparece en los prerrequisitos. La verificación en máquina limpia por un revisor externo es el único test que detecta esos gaps — y el checklist la hace obligatoria, no opcional.

El checklist de producción no es opcional — es el contrato del equipo consigo mismo sobre qué significa "listo para producción". Cada ítem que el equipo completa antes del despliegue es un incidente potencial que no ocurrirá en producción. Cada ítem que se omite por presión de tiempo es una deuda de confiabilidad que se paga con intereses en el primer incidente.

Para recordar: El valor del checklist de producción aumenta con cada incidente post-despliegue que hubiera prevenido. Si un incidente ocurre y el checklist lo habría detectado, el checklist necesita un ítem adicional — o el proceso que garantiza que el ítem existente no se omite bajo presión de tiempo necesita reforzarse.

Módulo 12 – Capítulo 10 – Sección 03

Lecciones aprendidas: reflexión sobre las decisiones técnicas tomadas durante el proyecto

Las lecciones aprendidas de un proyecto de ingeniería de IA tienen un valor que trasciende el proyecto en el que se generaron. El código puede reescribirse; las decisiones mal tomadas sin documentar se repetirán en el siguiente proyecto. Esta sección no es un inventario de errores — es una destilación de los insights técnicos que emergen cuando se construye un sistema completo bajo restricciones reales de calidad, latencia y costo, y se enfrenta el gap entre la intuición inicial y lo que los datos mostraron.

La primera lección — y la más contraintuitiva para ingenieros con experiencia previa en búsqueda tradicional — es que **los parámetros por defecto de los frameworks de chunking rara vez son óptimos para un dominio específico**. La intuición inicial del equipo era que un chunk más grande (1024 tokens) retendría más contexto y produciría mejor faithfulness. Los datos contradijeron esa intuición con claridad: el experimento de ablación con el golden dataset de 200 pares mostró que 1024 tokens producía un context precision de 0.71, mientras que 512 tokens con 64 de overlap producía 0.80. La diferencia no es marginal — es la diferencia entre un sistema que frecuentemente incluye contexto irrelevante en el prompt y uno que lo filtra efectivamente. La causa técnica es que los documentos del corpus (documentación técnica de ingeniería de software) tienen una densidad de información alta: 512 tokens captura una idea completa en la mayoría de los casos, mientras que 1024 tokens tiende a mezclar ideas de secciones consecutivas que el modelo de embedding no puede separar semánticamente.

La segunda lección es que **el reranker justifica ampliamente su latencia adicional**. El experimento de ablación sin reranker fue solicitado por el equipo de producto preocupado por el impacto del reranker en la latencia P95: 280ms adicionales representan casi el 10% del presupuesto de latencia. El experimento mostró que eliminar el reranker mejoraba la latencia P95 de 2800ms a 2520ms pero degradaba el faithfulness de 0.87 a 0.74 — una caída de 13 puntos que convierte respuestas fiables en respuestas frecuentemente imprecisas. El equipo de producto retiró inmediatamente la propuesta de eliminar el reranker; la lección es que la

latencia de un componente debe evaluarse en el contexto del impacto en calidad que su eliminación produce, no de forma aislada.

La tercera lección es sobre los ADRs: **las decisiones que se toman bajo presión de tiempo son exactamente las que más importa documentar**. El ADR-003 sobre el diseño del agente (LangGraph sobre AgentExecutor) se escribió durante la semana de diseño inicial, con tiempo suficiente para articular el razonamiento. El ADR sobre la estrategia de re-indexación de documentos actualizados se tomó durante una emergencia operativa en la semana 6, bajo presión, y no se documentó en el momento. Seis semanas después, cuando el equipo necesitó revisar esa estrategia, la reconstrucción del razonamiento original tomó medio día de discusión. El tiempo no invertido en documentar la decisión bajo presión se multiplicó por diez en el tiempo de reconstrucción posterior.

La cuarta lección toca la arquitectura del testing agéntico: **los escenarios de fallo del agente deben diseñarse desde el primer día, no añadirse al final del proyecto**. Los tests unitarios de las herramientas individuales se escribieron junto con las herramientas; los tests de comportamiento del agente ante retrieval vacío, timeout del LLM y ambigüedad de consulta se diseñaron en la semana 8, cuando el sistema ya era complejo. El resultado fue que los tests de comportamiento encontraron tres bugs de manejo de errores en el grafo LangGraph que habían estado presentes desde la semana 3 — bugs que habrían sido triviales de corregir en el momento y que requirieron refactoring no trivial a las 8 semanas.

Lecciones aprendidas por componente

- **Chunking:** los parámetros por defecto (1024/200) son el punto de partida, nunca la configuración óptima; el experimento de ablación con RAGAS en el golden dataset es la única forma objetiva de encontrar los parámetros correctos para el corpus específico.
- **Reranking:** la latencia adicional del reranker (280ms) es pequeña comparada con la degradación de faithfulness de 13 puntos que produce su eliminación — el reranker no es un componente de optimización de rendimiento sino una pieza de calidad esencial del pipeline.
- **ADRs bajo presión:** las decisiones tomadas en situaciones de emergencia operativa son las que más impactan el sistema a largo plazo y las que más cuesta reconstruir sin documentación — el ADR debe escribirse en el mismo día de la decisión, aunque sea en borrador.
- **Testing agéntico desde el inicio:** los escenarios de fallo del agente (retrieval vacío, timeout, max_iterations alcanzado, ambigüedad de consulta) deben diseñarse cuando se

diseña el agente, no cuando el sistema está completo — son los que revelan los bugs de manejo de errores más costosos de corregir tarde.

- **Instrumentación temprana:** añadir los spans de OpenTelemetry a un sistema ya construido es significativamente más costoso que instrumentar desde el primer commit; en la práctica, la instrumentación tardía tiende a ser incompleta porque el equipo no puede justificar el costo de instrumentar exhaustivamente un sistema que ya funciona.

Nota del Arquitecto: *La lección sobre instrumentación temprana merece una elaboración práctica. El argumento común para instrumentar más tarde es "primero hacemos que funcione, luego añadimos observabilidad". El problema es que un sistema que funciona sin observabilidad es un sistema cuya operabilidad nadie ha medido. La instrumentación temprana no solo produce observabilidad antes — cambia cómo el equipo diseña el sistema. Cuando el primer span de OpenTelemetry está en el código desde el inicio, el equipo empieza a diseñar las fases del pipeline como unidades de medición (dónde empieza cada span, qué atributos se adjuntan), no solo como unidades de código. Este cambio de perspectiva produce sistemas más observables desde el diseño, no solo desde la implementación.*

Las lecciones aprendidas de un proyecto son más útiles cuando se comparten con el equipo antes del siguiente proyecto, no después. La práctica del post-mortem al cierre de cada proyecto —un documento estructurado con las cinco lecciones más valiosas, su causa técnica y la recomendación concreta para el siguiente proyecto— es lo que convierte el aprendizaje individual en aprendizaje organizacional.

Para recordar: Las lecciones aprendidas del proyecto son el activo intangible más valioso que el equipo produce — el código puede reescribirse, pero las decisiones mal tomadas sin documentar se repetirán invariablemente en el siguiente proyecto del equipo.

Módulo 12 – Capítulo 10 – Sección 04

Extensiones posibles: cómo seguir evolucionando el sistema más allá del alcance del proyecto

El sistema integrador del proyecto final es una plataforma con capacidades de producción demostradas, no un prototipo. Tiene la arquitectura, la instrumentación y los controles que permiten evolucionar hacia capacidades más avanzadas sin reescritura estructural: el pipeline de ingesta es extensible mediante la interfaz `BaseDocumentParser`, el agente acepta nuevas herramientas mediante el decorador `@tool`, la base vectorial puede reconvertirse con un proceso blue-green documentado, y el pipeline CI/CD propaga automáticamente cualquier cambio con el gate de evaluación RAGAS activo como guardia de regresión. Las extensiones que se describen a continuación no son aspiraciones teóricas — son los siguientes pasos naturales del sistema, priorizados por impacto esperado sobre el caso de uso actual.

La extensión de mayor impacto a corto plazo es el **fine-tuning del modelo de embedding sobre el corpus técnico del dominio**. El modelo `text-embedding-3-small` de OpenAI es un modelo de embeddings de propósito general entrenado sobre un corpus masivo y diverso — lo que significa que su representación semántica del espacio latente no está optimizada para la terminología específica del corpus de documentación técnica de ingeniería de software del sistema. Términos como "HNSW", "scalar quantization", "payload index" o "ReAct" pueden ser representados de forma subóptima porque tienen baja frecuencia en el corpus de preentrenamiento relativa a su frecuencia en el corpus de producción del sistema. El fine-tuning con contrastive learning sobre pares (query, documento relevante) del golden dataset puede mejorar el recall@5 entre 0.05 y 0.12 puntos para consultas que usen terminología específica del dominio. La inversión es significativa (días de ingeniería para el pipeline de fine-tuning + costo de compute) y debe justificarse con el análisis de las categorías de queries donde el retrieval actual tiene bajo recall, antes de comenzar.

La segunda extensión es el **soporte multimodal para documentos con contenido visual**. La documentación técnica de ingeniería de software frecuentemente contiene diagramas de arquitectura, capturas de dashboards, tablas de métricas y pseudocódigo en imágenes. El sistema actual no puede indexar este contenido porque `text-embedding-3-`

`small` no procesa imágenes. La extensión requiere un pipeline de preprocesamiento: detectar imágenes en los documentos durante la ingesta, enviarlas a GPT-4o Vision o Claude 3.5 Sonnet Anthropic con un prompt que solicita una descripción textual densa del contenido visual (no una descripción general — una descripción que capture los valores numéricos, las relaciones entre componentes, y el significado técnico del diagrama), y añadir esa descripción como un chunk de texto adicional asociado al documento fuente. Este enfoque no requiere cambios en el schema de Qdrant ni en el modelo de embedding.

La tercera extensión — **sistema de feedback activo con captura de ratings** — cierra el ciclo entre el comportamiento del sistema en producción y la mejora del modelo. Actualmente, el sistema produce respuestas pero no tiene un mecanismo para saber qué respuestas fueron útiles para el usuario. Añadir un endpoint `POST /feedback` que acepte `{session_id, message_id, rating: 1|-1, comment: Optional[str]}` permite acumular un dataset de preferencias: pares (pregunta, respuesta, rating) que representan la señal directa del usuario sobre la calidad del sistema. Estos datos pueden usarse para: identificar las categorías de queries donde el sistema tiene peor desempeño (análisis de ratings negativos agrupados por cluster de queries), mejorar el golden dataset con los pares de alta calidad positiva (las respuestas con rating +1 son candidatos a golden pairs), y eventualmente para DPO (Direct Preference Optimization) si se acumulan suficientes pares contrastivos (misma query, respuesta positiva vs respuesta negativa).

Extensiones priorizadas por impacto

- **Fine-tuning de embedding (impacto: alto, plazo: 4-6 semanas):** contrastive learning sobre pares del golden dataset para mejorar recall@5 en terminología específica del dominio; prerequisite: análisis de recall por categoría para confirmar que hay un gap de terminología significativo.
- **Soporte multimodal (impacto: medio-alto, plazo: 2-3 semanas):** pipeline OCR + descripción GPT-4o Vision para indexar diagramas, tablas y capturas de dashboards como chunks de texto adicionales; sin cambios en el schema de Qdrant.
- **Feedback activo con ratings (impacto: medio, plazo: 1-2 semanas):** endpoint `POST /feedback`, dashboard de ratings por categoría, pipeline de curación de golden pairs desde feedback positivo; base para DPO a largo plazo.
- **Multi-tenancy (impacto: alto si hay expansión de usuarios, plazo: 4-8 semanas):** aislamiento por tenant en Qdrant (colecciones separadas para nivel de aislamiento alto, o particionamiento por payload para nivel de aislamiento medio), RBAC por organización, billing por tenant con atribución de costo.

- **Agent memory de largo plazo (impacto: medio, plazo: 3-4 semanas):** base de datos de hechos aprendidos de conversaciones anteriores (PostgreSQL con pgvector para búsqueda semántica sobre el historial de conversaciones del usuario), inyección selectiva de contexto relevante histórico en cada nueva conversación.

Nota del Arquitecto: La extensión de multi-tenancy merece una advertencia sobre su complejidad subestimada. La primera implementación de multi-tenancy que los equipos proponen es "añadir un campo `tenant_id` a los documentos en Qdrant y filtrar por él". Esa implementación funciona para uso interno con un conjunto pequeño de tenants de confianza mutua — pero no es multi-tenancy real porque no garantiza el aislamiento de datos entre tenants. Un bug en el filtro de `tenant_id` (un campo nulo, una condición de race condition en una transacción) puede exponer documentos de un tenant a otro. El multi-tenancy correcto con aislamiento garantizado requiere colecciones separadas en Qdrant por tenant — lo que añade complejidad operativa (más colecciones que monitorear, más índices que mantener) pero garantiza que ningún bug de aplicación puede violar el aislamiento. La decisión entre ambos enfoques es un ADR que debe documentarse antes de comenzar la implementación.

Las extensiones posibles del sistema deben priorizarse con el mismo rigor que el proyecto inicial — no como ideas interesantes sino como inversiones con impacto esperado cuantificado, costo de implementación estimado, y riesgo de aumento de complejidad del sistema. La práctica de escribir un ADR antes de comenzar cada extensión significativa no es overhead burocrático — es la diferencia entre extensiones que el sistema puede absorber y extensiones que lo hacen más difícil de operar que antes de añadirlas.

Para recordar: Las extensiones posibles del sistema deben priorizarse con el mismo rigor que el proyecto inicial: impacto esperado cuantificado, costo de implementación estimado, y documentación en un nuevo ADR antes de comenzar. Una extensión sin ADR es una decisión sin registro.

Módulo 12 – Capítulo 10 – Sección 05

El AI Engineer que fuiste y el AI Engineer que eres: síntesis del recorrido del libro

Al comenzar el libro, la visión del trabajo del AI Engineer era la de quien hace que los modelos funcionen: el ingeniero que elige el prompt correcto, conecta la API correcta, y produce en un notebook una respuesta que impresiona a quien la ve. Esa visión no era incorrecta — era incompleta. Lo que faltaba no era conocimiento técnico adicional sobre LLMs; era la comprensión de lo que significa construir un sistema de IA que no solo impresiona en una demo sino que funciona en producción durante semanas y meses, se degrada de forma controlada cuando algo falla, se evalúa con criterios que no son "se ve bien" sino "faithfulness ≥ 0.85 sobre un golden dataset de 200 pares con IAA $> 80\%$ ", y puede ser operado por cualquier ingeniero del equipo, no solo por quien lo construyó. El recorrido del libro fue el trayecto entre esas dos visiones.

El recorrido comenzó con los fundamentos que explican por qué los modelos de lenguaje hacen lo que hacen: la arquitectura transformer, los mecanismos de atención, la tokenización, el espacio latente de los embeddings. No como conocimiento de relleno antes de llegar al "código real" — sino como el lenguaje que permite razonar sobre el comportamiento de los modelos cuando se desvían de lo esperado. Sin ese lenguaje, los problemas de hallucination, de context poisoning, de prompt leakage, o de degradación semántica en la búsqueda vectorial se diagnostican por ensayo y error. Con él, se diagnostican por análisis causal. La diferencia entre ambas formas de trabajo no es de velocidad — es de rigor.

La ingeniería de prompts reveló que los prompts son código con un contrato implícito que puede hacerse explícito: qué comportamiento se especifica, qué comportamiento se deja a la discretion del modelo, y cómo verificar que el modelo efectivamente cumple el contrato. El sistema prompt del agente de este módulo es la manifestación de ese aprendizaje: no un texto largo que intenta anticipar todo, sino un conjunto de instrucciones precisas con los límites del comportamiento del agente documentados con la misma precisión con la que se documentan los límites de una función con sus precondiciones y postcondiciones.

El módulo de RAG demostró que la búsqueda semántica no es búsqueda exacta con palabras distintas — es un problema de recuperación de información con sus propias métricas (recall@k, context precision, faithfulness), sus propias técnicas de optimización (chunking, embeddings, índices HNSW), y sus propias formas de fallo (retrieval relevante pero no suficiente, retrieval de alta precision pero baja cobertura, retrieval que distrae al LLM en lugar de ayudarlo). El golden dataset de 200 pares con Inter-Annotator Agreement > 80% no es un artefacto burocrático — es la herramienta que hace posible medir ese comportamiento de forma objetiva y mejorarlo de forma dirigida.

Los agentes expandieron la visión más allá del pipeline estático. El patrón ReAct demostró que razonar y actuar de forma iterativa sobre el resultado de las acciones producía sistemas capaces de resolver problemas que ningún pipeline de paso único podía abordar. LangGraph reveló que el estado del agente no es una variable secundaria sino un ciudadano de primera clase del diseño: qué información persiste entre interacciones, qué se olvida deliberadamente, y cómo el agente recibe el contexto de la conversación previa son decisiones de diseño con las mismas consecuencias que elegir el schema de la base de datos. Y el testing agéntico enseñó que los escenarios de fallo — retrieval vacío, timeout del LLM, iteraciones que no convergen — son los escenarios que más importa probar porque son los que el usuario experimentará en los momentos de mayor frustración.

La seguridad introdujo una dimensión que ningún benchmark de calidad o rendimiento mide: la robustez ante usuarios adversariales. El STRIDE threat model sobre el sistema RAG+agente no fue un ejercicio académico — fue el análisis que reveló que el sistema tenía vectores de ataque específicos de los sistemas de IA (prompt injection indirecto a través de documentos, exfiltración de contexto del sistema prompt) que los controles de seguridad de los sistemas web convencionales no cubren. El red teaming de 50 ataques con tasa de bypass final del 2% no fue el final de la historia de seguridad del sistema — fue el inicio: el punto desde el cual el monitoreo de production puede detectar regresiones.

El MLOps transformó el sistema de un artefacto que funciona en la máquina del ingeniero a un sistema que puede desplegarse reproduciblemente, monitorearse automáticamente y actualizarse sin downtime. El Dockerfile multi-stage, la imagen sin usuario root, el pipeline CI/CD con gate de evaluación RAGAS, el Terraform para la infraestructura, el Argo Rollouts para el canary — cada uno de estos artefactos tiene un costo de implementación que se recupera en el primer despliegue que no produce un incidente post-actualización, en el primer rollback automatizado que detecta una regresión antes de que afecte al 100% del tráfico. La ingeniería de costos — el cost dashboard por equipo, las alertas de anomalía de costo, el

desglose de costo por request — completó la imagen: los sistemas de IA tienen un perfil de costo operativo que los sistemas de software convencionales no tienen, y gestionarlo con la misma rigurosidad que la latencia o el error rate es parte del trabajo del AI Engineer en producción.

La observabilidad cerró el ciclo entre lo que el sistema hace y lo que el equipo sabe que el sistema hace. El tracing distribuido con OpenTelemetry no mide si el sistema funciona — mide cómo funciona: cuánto tiempo toma cada etapa del pipeline, cuántas iteraciones necesita el agente en promedio, cuánto cuesta cada request, y si la calidad observada en producción está derivando de los umbrales establecidos con el golden dataset. El drift monitoring —análisis de reranking scores, clustering de queries de producción, evaluación semanal del golden dataset— completó la imagen de la observabilidad de IA: la degradación de calidad en sistemas de IA no es un evento discreto que dispara una alerta, es un proceso gradual que solo el monitoreo continuo puede detectar antes de que sea visible para los usuarios.

La documentación técnica del Capítulo 9 convirtió todo ese conocimiento acumulado en un sistema que puede sobrevivir a la rotación del equipo, a la incorporación de nuevos integrantes, y al paso del tiempo. Un README que permite el setup en menos de 30 minutos, un runbook que permite resolver el incidente más frecuente sin conocimiento previo del sistema, una guía de contribución que permite añadir una nueva herramienta al agente en medio día — no son documentos periféricos al sistema. Son parte del sistema: la capa que hace que el sistema pueda ser mantenido por un equipo, no solo por la persona que lo construyó.

La competencia que el recorrido construyó

El AI Engineer que completó este libro puede ahora:

- **Diseñar con criterios explícitos:** definir los umbrales de éxito del sistema antes de escribir código, construir el golden dataset que los mide con rigor estadístico, y usar esos datos para tomar decisiones de diseño justificadas con evidencia en lugar de con intuición.
- **Implementar con profundidad técnica:** construir un pipeline RAG con búsqueda híbrida, reranking, chunking validado, y evaluación RAGAS; implementar un agente ReAct con LangGraph con estado persistente, herramientas tipadas, y testing adversarial; aplicar controles de seguridad con STRIDE como marco y red teaming como validación.
- **Operar con responsabilidad:** desplegar con Dockerfile seguro y pipeline CI/CD con gate de evaluación; monitorear con OTel, Grafana y alertas con runbooks; detectar drift de

calidad antes de que sea visible para los usuarios; gestionar el costo operativo como un SLA de primera clase.

- **Documentar como práctica de ingeniería:** producir artefactos de documentación que convierten el conocimiento del equipo en una propiedad del sistema — README, OpenAPI, runbook, threat model, guía de contribución — y mantenerlos como ciudadanos de primera clase del repositorio.
- **Decidir con registro:** usar ADRs para documentar no solo qué se decidió sino por qué, con qué información, y con qué compensaciones — convirtiendo las decisiones técnicas en conocimiento organizacional que persiste más allá del equipo que las tomó.

Nota del Arquitecto: Hay un momento en la carrera de un ingeniero de software en el que la relación con la incertidumbre cambia. Al principio, la incertidumbre sobre si el sistema funcionará en producción produce ansiedad — la sensación de que hay demasiadas variables fuera del control para garantizar el resultado. Con el tiempo y la práctica, esa incertidumbre se convierte en el material de trabajo natural: el sistema de IA que se comporta de forma diferente ante queries que el golden dataset no cubría no es un fracaso del diseño — es la observación que guía la siguiente iteración. El AI Engineer maduro no es el que elimina la incertidumbre — es el que ha construido los instrumentos de medición (golden dataset, RAGAS, drift monitoring) que le permiten trabajar con la incertidumbre de forma productiva, detectar cuándo se convierte en un problema antes de que sea crítico, y mejorar el sistema de forma dirigida. Ese es el cambio más profundo que este libro intenta producir: no solo el conocimiento de las herramientas, sino la postura hacia el trabajo de ingeniería de IA en producción.

La diferencia entre el AI Engineer que fuiste al comenzar este libro y el AI Engineer que eres ahora no es una lista de herramientas aprendidas — es una forma de pensar sobre los sistemas de IA. Una forma que sabe que el modelo es solo una pieza del sistema, que la calidad requiere medición continua con criterios explícitos, que la seguridad no es un módulo que se añade al final sino una dimensión del diseño desde el principio, y que un sistema que no puede ser operado, extendido y documentado por el equipo completo no es un sistema en producción — es una deuda técnica en espera de crisis.

El proyecto final no fue el destino. Fue el primer sistema de IA que este ingeniero construyó con el rigor que los sistemas en producción requieren. El siguiente será mejor — porque habrá

el golden dataset del anterior como referencia, los ADRs del anterior como memoria, y las lecciones aprendidas del anterior como brújula.

Para recordar: La diferencia entre un ingeniero que usa IA y un AI Engineer es la capacidad de construir sistemas de IA que puedan evaluarse, operarse y mejorarse de forma continua — y esa capacidad es exactamente lo que el proyecto final demostró que es posible.

Módulo 12 – Capítulo 10 – Sección 06

Cierre: la ingeniería de IA no termina con el despliegue — comienza con él

El sistema integrador que los diez capítulos de este módulo construyeron paso a paso — desde los ADRs de la semana de diseño hasta el runbook del incidente más frecuente, desde el chunking validado con RAGAS hasta las alertas con periodo de pending de 3 minutos — es ahora un sistema en producción. Ese es el punto de llegada del proyecto. Y es, simultáneamente, el punto de partida de la ingeniería de IA real.

En producción, el sistema enfrenta un régimen que ningún entorno de desarrollo puede reproducir completamente. Los usuarios formulan preguntas sobre combinaciones de temas que el golden dataset no cubría. Los documentos en la base de conocimiento envejecen mientras el sistema sigue sirviéndolos con la misma confianza que cuando se indexaron. El proveedor del LLM actualiza el modelo con mejoras de seguridad que cambian sutilmente el comportamiento en dimensiones que el sistema prompt no anticipó. Un patrón de uso nuevo — sesiones largas con muchas iteraciones agénticas — produce un perfil de costo que los modelos de estimación del proyecto subestimaron. Y tarde o temprano, un ataque de un tipo que el red teaming no cubrió alcanza los controles del sistema y produce el primer incidente de seguridad.

La ingeniería de IA en producción es la práctica de responder a esos eventos con los instrumentos que el sistema ya tiene: las trazas de OpenTelemetry que muestran exactamente qué etapa del pipeline produjo la respuesta inesperada, el golden dataset que permite medir si el comportamiento observado es una regresión o una variación dentro del rango normal, los ADRs que explican por qué el sistema fue diseñado de la forma en que fue diseñado y qué restricciones se aceptaron conscientemente, el runbook que guía la resolución del incidente sin depender de la persona que construyó el sistema. Estos instrumentos no existen en el sistema porque eran políticamente convenientes — existen porque la experiencia de operar sistemas de IA en producción ha demostrado repetidamente que sin ellos, la degradación gradual de la calidad no se detecta hasta que es visible para los usuarios, los incidentes de

producción duran horas en lugar de minutos, y el conocimiento del sistema muere cuando el ingeniero fundador deja el proyecto.

La síntesis que el Capítulo 10-05 articuló — el recorrido desde usar LLMs hasta operar sistemas de IA evaluables — no es el cierre de un aprendizaje. Es la descripción de la competencia que permite seguir aprendiendo de forma dirigida. El AI Engineer que tiene un sistema en producción con observabilidad, evaluación continua y documentación viva tiene algo que el AI Engineer con un prototipo en un notebook no tiene: datos reales del comportamiento del sistema ante usuarios reales. Esos datos son la materia prima del siguiente ciclo de mejora: el análisis de los casos donde faithfulness cayó por debajo del umbral revela qué tipos de documentos producen poor grounding; el clustering de queries de producción revela qué categorías de preguntas el golden dataset no cubre; el análisis de ratings negativos (si el sistema de feedback activo está implementado) revela exactamente qué respuestas los usuarios encontraron insuficientes y por qué.

Lo que el Capítulo 10 integra

- **Rúbrica cuantitativa:** seis dimensiones con criterios verificables — faithfulness ≥ 0.82 , task completion $\geq 75\%$, bypass $< 5\%$ — diseñados para ser verificados con scripts automatizados, no con juicio subjetivo; cierre del ciclo abierto en la Sección 01-05 del módulo.
- **Checklist de producción:** 30 verificaciones en seis categorías que convierten la pregunta "¿está listo para producción?" en una lista de condiciones verificables; la ansiedad del primer despliegue se gestiona con procedimiento, no con confianza.
- **Lecciones aprendidas:** cinco insights técnicos sobre chunking, reranking, ADRs, testing agéntico e instrumentación temprana — destilados de las decisiones concretas del proyecto para que el siguiente proyecto no las repita como experimentos.
- **Extensiones posibles:** cinco caminos de evolución priorizados por impacto — fine-tuning de embeddings, soporte multimodal, feedback activo, multi-tenancy, agent memory — cada uno con la estructura de un ADR pendiente de escribir.
- **Síntesis del recorrido:** el cambio de perspectiva entre el ingeniero que usa modelos de IA y el AI Engineer que construye sistemas de IA evaluables en producción — articulado como postura, no como lista de herramientas dominadas.

El primer despliegue a producción de un sistema de IA no es el punto final del proyecto. Es el punto en el que el sistema comienza a generar las observaciones reales que permiten

mejorarlo con evidencia en lugar de con suposiciones. Es el momento en que el trabajo de ingeniería de IA — el trabajo real, el que importa — comienza.

"The best way to learn how to build AI systems is to build AI systems, run them in production, and pay attention to what breaks." — Andrej Karpathy